

八位十进制计数器设计

1、实验任务：

设计 6 位十进制计数显示器电路，要求所设计的电路具有以下功能：

- 1 能够实现 0-99999999 的计数并显示；
- 2 具备复位清零功能。

2、实验原理：

前面我们学习了 10 进制计数器，如果我们再在每个计数器加一个溢出位，信号接下一个计数器，这样我们连接 6 个，时钟输入 50MHz 分频 0.1Hz，图 1.1 是设计示意图。

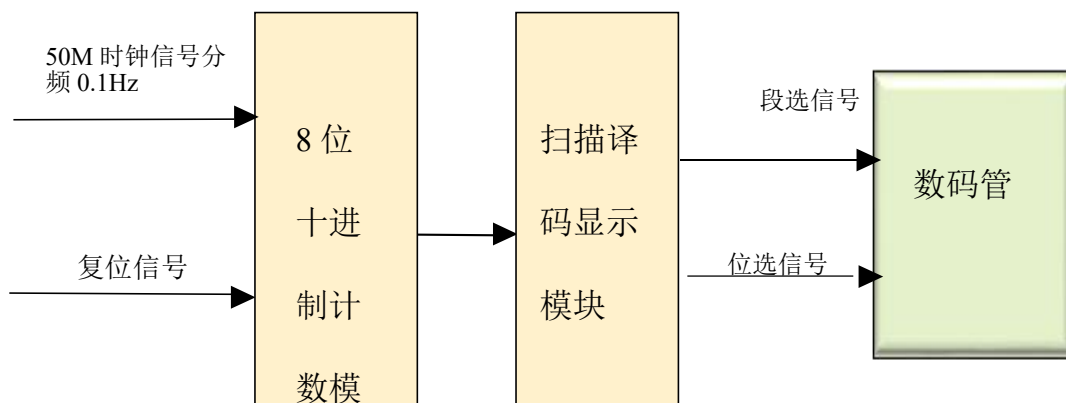


图 1.1

3、实验程序：

实验程序供三个：顶层模块、计数模块及数码扫描译码模块
顶层：

```
module top_seg_led(  
    //global clock  
    input CLOCK, // 全局时钟信号  
    input rst, // 复位信号（低有效）  
    //seg_led interface  
    output [7:0] SEL, // 数码管位选信号  
    output [7:0] DIG // 数码管段选信号  
);
```

```
//wire define
wire [19:0] data; // 数码管显示的数值
wire [ 5:0] point; // 数码管小数点的位置
wire en; // 数码管显示使能信号
wire sign; // 数码管显示数据的符号位
assign SEL[7:6] = 2'b11;
//计数器模块，产生数码管需要显示的数据
count u_count(
.clk (CLOCK ), // 时钟信号
.rst_n (rst), // 复位信号
.data (data ), // 6 位数码管要显示的数值
.point (point ), // 小数点具体显示的位置,高电平有效
.en (en ), // 数码管使能信号
.sign (sign ) // 符号位
);
//数码管动态显示模块
seg_led u_seg_led(
.clk (CLOCK ), // 时钟信号
.rst_n (rst), // 复位信号
.data (data ), // 显示的数值
.point (point ), // 小数点具体显示的位置,高电平有效
.en (en ), // 数码管使能信号
.sign (sign ), // 符号位，高电平显示负号(-)
.seg_sel (SEL[5:0] ), // 位选
.seg_led (DIG ) // 段选
);
Endmodule
计数：
module count(
//module clock
input clk , // 时钟信号
input rst_n, // 复位信号
//user interface
output reg [19:0] data , // 6 个数码管要显示的数值
output reg [ 5:0] point, // 小数点的位置,高电平点亮对应数码管位上的小数点
output reg en , // 数码管使能信号
output reg sign // 符号位，高电平时显示负号，低电平不显示负号
```

```
);  
//parameter define  
parameter MAX_NUM = 23'd5000_000; // 计数器计数的最大值  
//reg define  
reg [22:0] cnt ; // 计数器，用于计时 100ms  
reg flag; // 标志信号  
//计数器对系统时钟计数达 10ms 时，输出一个时钟周期的脉冲信号  
always @ (posedge clk or negedge rst_n) begin  
if (!rst_n) begin  
cnt <= 23'b0;  
flag <= 1'b0;  
end  
else if (cnt < MAX_NUM - 1'b1) begin  
cnt <= cnt + 1'b1;  
flag <= 1'b0;  
end  
else begin  
cnt <= 23'b0;  
flag <= 1'b1;  
end  
end  
//数码管需要显示的数据，从 0 累加到 999999  
always @ (posedge clk or negedge rst_n) begin  
if (!rst_n)begin  
data <= 20'b0;  
point <= 6'b000000;  
en <= 1'b0;  
sign <= 1'b0;  
end  
else begin  
point <= 6'b000000; //不显示小数点  
en <= 1'b1; //打开数码管使能信号  
sign <= 1'b0; //不显示负号  
if (flag) begin //显示数值每隔 0.01s 累加一次  
if(data < 20'd999999)  
data <= data + 1'b1;  
else  
data <= 20'b0;
```

```

end
end
end
endmodule
数码扫描译码
module seg_led(
input clk ,// 时钟信号
input rst_n ,// 复位信号
input [19:0] data ,// 6 位数码管要显示的数值
input [5:0] point ,// 小数点具体显示的位置,从高到低,高电平有效
input en ,// 数码管使能信号
input sign ,// 符号位 (高电平显示"-")号)
output reg [5:0] seg_sel, // 数码管位选, 最左侧数码管为最高位
output reg [7:0] seg_led // 数码管段选
);
//parameter define
localparam CLK_DIVIDE = 4'd10 ;// 时钟分频系数
localparam MAX_NUM = 13'd5000 ;// 对数码管驱动时钟(5MHz)计数 1ms 所需的计
数值

//reg define
reg [ 3:0] clk_cnt ;// 时钟分频计数器
reg dri_clk ;// 数码管的驱动时钟,5MHz
reg [23:0] num ;// 24 位 bcd 码寄存器
reg [12:0] cnt0 ;// 数码管驱动时钟计数器
reg flag ;// 标志信号 (标志着 cnt0 计数达 1ms)
reg [2:0] cnt_sel ;// 数码管位选计数器
reg [3:0] num_disp ;// 当前数码管显示的数据
reg dot_disp ;// 当前数码管显示的小数点
//wire define
wire [3:0] data0 ;// 十万位数
wire [3:0] data1 ;// 万位数
wire [3:0] data2 ;// 千位数
wire [3:0] data3 ;// 百位数
wire [3:0] data4 ;// 十位数
wire [3:0] data5 ;// 个位数
//提取显示数值所对应的十进制数的各个位
assign data0 = data % 4'd10; // 个位数

```

```

assign data1 = data / 4'd10 % 4'd10 ; // 十位数
assign data2 = data / 7'd100 % 4'd10 ; // 百位数
assign data3 = data / 10'd1000 % 4'd10 ; // 千位数
assign data4 = data / 14'd10000 % 4'd10; // 万位数
assign data5 = data / 17'd100000; // 十万位数
//对系统时钟 10 分频，得到的频率为 5MHz 的数码管驱动时钟 dri_clk
always @(posedge clk or negedge rst_n) begin
if(!rst_n) begin
clk_cnt <= 4'd0;
dri_clk <= 1'b1;
end
else if(clk_cnt == CLK_DIVIDE/2 - 1'd1) begin
clk_cnt <= 4'd0;
dri_clk <= ~dri_clk;
end
else begin
clk_cnt <= clk_cnt + 1'b1;
dri_clk <= dri_clk;
end
end
//将 20 位 2 进制数转换为 8421bcd 码(即使用 4 位二进制数表示 1 位十进制数)
always @(posedge dri_clk or negedge rst_n) begin
if(!rst_n)
num <= 24'b0;
else begin
if(data5 || point[5]) begin //如果显示数据为 6 位十进制数，
num[23:20] <= data5; //则依次给 6 位数码管赋值
num[19:16] <= data4;
num[15:12] <= data3;
num[11:8] <= data2;
num[7:4] <= data1;
num[3:0] <= data0;
end
else begin
if(data4 || point[4]) begin //如果显示数据为 5 位十进制数，则给低 5 位数码管赋值
num[19:0] <= {data4,data3,data2,data1,data0};
if(sign)
num[23:20] <= 4'd11; //如果需要显示负号，则最高位（第 6 位）为符号位

```

```
else
num[23:20] <= 4'd10; //不需要显示负号时，则第 6 位不显示任何字符
end
else begin //如果显示数据为 4 位十进制数，则给低 4 位数码管赋值
if (data3 || point[3]) begin
num[15: 0] <= {data3,data2,data1,data0};
num[23:20] <= 4'd10; //第 6 位不显示任何字符
if(sign) //如果需要显示负号，则最高位（第 5 位）为符号位
num[19:16] <= 4'd11;
else //不需要显示负号时，则第 5 位不显示任何字符
num[19:16] <= 4'd10;
end
else begin //如果显示数据为 3 位十进制数，则给低 3 位数码管赋值
if (data2 || point[2]) begin
num[11: 0] <= {data2,data1,data0};
//第 6、5 位不显示任何字符
num[23:16] <= {2{4'd10}};
if(sign) //如果需要显示负号，则最高位（第 4 位）为符号位
num[15:12] <= 4'd11;
else //不需要显示负号时，则第 4 位不显示任何字符
num[15:12] <= 4'd10;
end
else begin //如果显示数据为 2 位十进制数，则给低 2 位数码管赋值
if (data1 || point[1]) begin
num[ 7: 0] <= {data1,data0};
//第 6、5、4 位不显示任何字符
num[23:12] <= {3{4'd10}};
if(sign) //如果需要显示负号，则最高位（第 3 位）为符号位
num[11:8] <= 4'd11;
else //不需要显示负号时，则第 3 位不显示任何字符
num[11:8] <= 4'd10;
end
else begin //如果显示数据为 1 位十进制数，则给最低位数码管赋值
num[3:0] <= data0;
//第 6、5 位不显示任何字符
num[23:8] <= {4{4'd10}};
if(sign) //如果需要显示负号，则最高位（第 2 位）为符号位
num[7:4] <= 4'd11;
```

```
else //不需要显示负号时，则第 2 位不显示任何字符
num[7:4] <= 4'd10;
end
end
end
end
end
end
end
end
//每当计数器对数码管驱动时钟计数时间达 1ms，输出一个时钟周期的脉冲信号
always @(posedge dri_clk or negedge rst_n) begin
if(rst_n == 1'b0) begin
cnt0 <= 13'b0;
flag <= 1'b0;
end
else if (cnt0 < MAX_NUM - 1'b1) begin
cnt0 <= cnt0 + 1'b1;
flag <= 1'b0;
end
else begin
cnt0 <= 13'b0;
flag <= 1'b1;
end
end
//cnt_sel 从 0 计数到 5，用于选择当前处于显示状态的数码管
always @(posedge dri_clk or negedge rst_n) begin
if(rst_n == 1'b0)
cnt_sel <= 3'b0;
else if(flag) begin
if(cnt_sel < 3'd5)
cnt_sel <= cnt_sel + 1'b1;
else
cnt_sel <= 3'b0;
end
else
cnt_sel <= cnt_sel;
end
end
```

```
//控制数码管位选信号，使 6 位数码管轮流显示
always @(posedge dri_clk or negedge rst_n) begin
if(!rst_n) begin
seg_sel <= 6'b111111; //位选信号低电平有效
num_disp <= 4'b0;
dot_disp <= 1'b1; //共阳极数码管，低电平导通
end
else begin
if(en) begin
case (cnt_sel)
3'd0 :begin
seg_sel <= 6'b111110; //显示数码管最低位
num_disp <= num[3:0] ; //显示的数据
dot_disp <= ~point[0]; //显示的小数点
end
3'd1 :begin
seg_sel <= 6'b111101; //显示数码管第 1 位
num_disp <= num[7:4] ;
dot_disp <= ~point[1];
end
3'd2 :begin
seg_sel <= 6'b111011; //显示数码管第 2 位
num_disp <= num[11:8];
dot_disp <= ~point[2];
end
3'd3 :begin
seg_sel <= 6'b110111; //显示数码管第 3 位
num_disp <= num[15:12];
dot_disp <= ~point[3];
end
3'd4 :begin
seg_sel <= 6'b101111; //显示数码管第 4 位
num_disp <= num[19:16];
dot_disp <= ~point[4];
end
3'd5 :begin
seg_sel <= 6'b011111; //显示数码管最高位
num_disp <= num[23:20];
```

```
dot_disp <= ~point[5];
end
default :begin
seg_sel <= 6'b111111;
num_disp <= 4'b0;
dot_disp <= 1'b1;
end
endcase
end
else begin
seg_sel <= 6'b111111; //使能信号为 0 时，所有数码管均不显示
num_disp <= 4'b0;
dot_disp <= 1'b1;
end
end
end
```

//控制数码管段选信号，显示字符

```
always @(posedge dri_clk or negedge rst_n) begin
if(!rst_n)
seg_led <= 8'hc0;
else begin
case (num_disp)
4'd0 : seg_led <= {dot_disp,7'b1000000}; //显示数字 0
4'd1 : seg_led <= {dot_disp,7'b1111001}; //显示数字 1
4'd2 : seg_led <= {dot_disp,7'b0100100}; //显示数字 2
4'd3 : seg_led <= {dot_disp,7'b0110000}; //显示数字 3
4'd4 : seg_led <= {dot_disp,7'b0011001}; //显示数字 4
4'd5 : seg_led <= {dot_disp,7'b0010010}; //显示数字 5
4'd6 : seg_led <= {dot_disp,7'b0000010}; //显示数字 6
4'd7 : seg_led <= {dot_disp,7'b1111000}; //显示数字 7
4'd8 : seg_led <= {dot_disp,7'b0000000}; //显示数字 8
4'd9 : seg_led <= {dot_disp,7'b0010000}; //显示数字 9
4'd10: seg_led <= 8'b1111111; //不显示任何字符
4'd11: seg_led <= 8'b1011111; //显示负号(-)
default:
seg_led <= {dot_disp,7'b1000000};
endcase
end
```

end

end

5、引脚约束:

#时钟周期约束

```
set_pin_assignment { CLOCK } { LOCATION = R7; IOSTANDARD = LVCMOS33; PULLTYPE =  
PULLUP; }
```

```
set_pin_assignment { DIG[0] } { LOCATION = C15; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[1] } { LOCATION = C16; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[2] } { LOCATION = B6; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[3] } { LOCATION = A5; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[4] } { LOCATION = B5; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[5] } { LOCATION = A8; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[6] } { LOCATION = A7; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { DIG[7] } { LOCATION = A6; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[0] } { LOCATION = D11; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[1] } { LOCATION = E11; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[2] } { LOCATION = F10; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[3] } { LOCATION = C13; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[4] } { LOCATION = E10; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[5] } { LOCATION = E12; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[6] } { LOCATION = E16; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
```

```
set_pin_assignment { SEL[7] } { LOCATION = F16; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { rst } { LOCATION = A2; IOSTANDARD = LVCMOS33; PULLTYPE =  
PULLUP; }
```

6、下载测试：

数码 1，十分之一秒计数时钟，满 10 进位。

